

Week 7

Database Design Pt. 2

The Design Process¹

1. **Determine the purpose of your database**
 - What is the high level purpose of this database?
2. **Find and organize the information required**
 - What information should be stored to fulfill the high level purpose?
3. **Divide the information into tables**
 - What are the entities to store?
4. **Turn information items into columns**
 - What are the individual data pieces that make up those entities (tables)?
5. **Specify primary keys**
 - What is the unique field (column) for each entity (table)?
6. **Set up the table relationships (and constraints)**
 - Do these tables relate? If so what is the connection?
7. **Refine your design**
 - Look for errors to inform improvements.
8. **Apply the normalization rules**
 - Review normalization rules and adjust as necessary.

¹ [Microsoft - DB Design Basics - The design process](#)

Keys - Specify Primary Keys

- Every table should include a column that uniquely identifies each row
- The unique column can be the **primary key** for the table
 - Primary key is referenced (by FK*) in **creating** the **links** (relationships) **between tables**
 - The primary key must be different in every row, there **cannot be duplicated values**
 - Primary keys **should not change** their value
- Primary Keys can be any column in your table, as long as they are unique
 - i. The key can be a column of data that is unique
 - ii. The key can also be a combination of columns that when combined make a unique value
 - iii. Typically creating an “id” column as the unique identifier is easiest

Primary Key

i.

number	name
n01234567	Auston Matthews

(Primary) Composite Key

ii.

code	instructor
HTTP5126-A	Matthew Bebis

Primary Key

iii.

bus_id	route
1	508

Relationships Between Tables

- With tables, columns, and unique IDs decided on... tables can be connected!
- Goals:
 - a. Identify where there are **relationships** between tables
 - b. Identify the relationship **type** between those tables

Finding Relationships

- Look for matching data in fields
 - This is often a field with the same name or data in different tables

instructor

name	birth_date
Matthew Bebis	1912-06-23

course

instructor	code	section	term
Matthew Bebis	HTTP5126	A	F24
Matthew Bebis	HTTP5126	B	F24
Matthew Bebis	HTTP5122	A	S24

- Determine if a table **HAS A** entity that is represented by another table
 - Example above: a course **HAS AN** instructor OR an instructor **HAS A** course to teach

Relationship Types

- When a relationship is found, it will have a relationship type
- **Relationships** will fall into 1 of 3 categories
 - a. one-to-one relationship
 - b. one-to-many relationship
 - c. many-to-many relationship

one-to-one relationship

country

name	population	capital_city
Canada	38.93 million	Ottawa
Denmark	5.9 million	Copenhagen

city

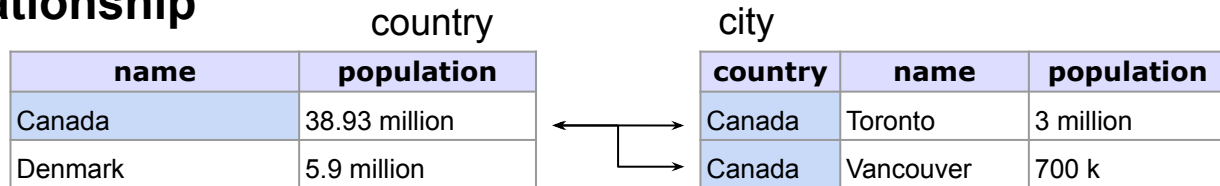
name	population
Ottawa	1 million
Copenhagen	600 k



- 1 row in a table connects to exactly 1 row in a different table
 - Example above: A country **HAS A** capital city

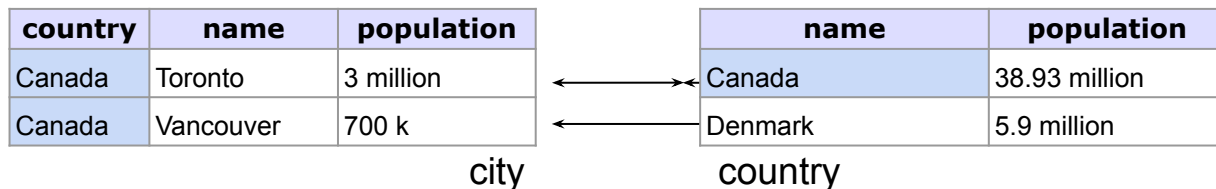
Relationship Types

one-to-many relationship



- 1 row in a table connects to many different rows in a different table
- OR
- Many rows in a table connect to 1 row in a different table
 - Example above: A country **HAS MANY** cities, a city **HAS A** country

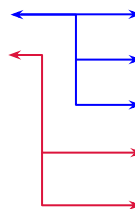
many-to-one relationship *same as one-to-many



Relationship Types

one-to-many relationship / Example

name	population
Canada	38.93 million
Denmark	5.9 million

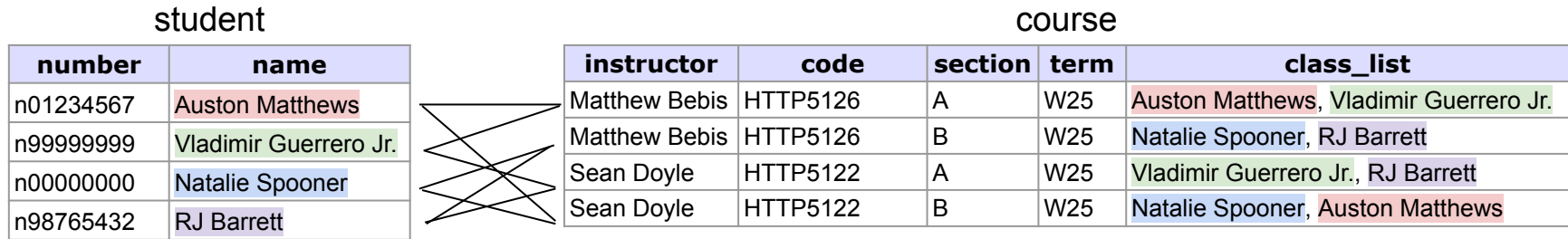


country	name	population
Canada	Toronto	3 million
Canada	Vancouver	700 k
Canada	Ottawa	1 million
Denmark	Copenhagen	600 k
Denmark	Odense	180 k

- Each **ONE** country **HAS MANY** cities
 - Canada has many cities
 - Denmark has many cities
- The **MANY** cities **HAVE** just **ONE** country
 - Toronto, Vancouver, and Ottawa all belong to Canada
 - Copenhagen and Odense all belong to Denmark

Relationship Types

many-to-many relationship

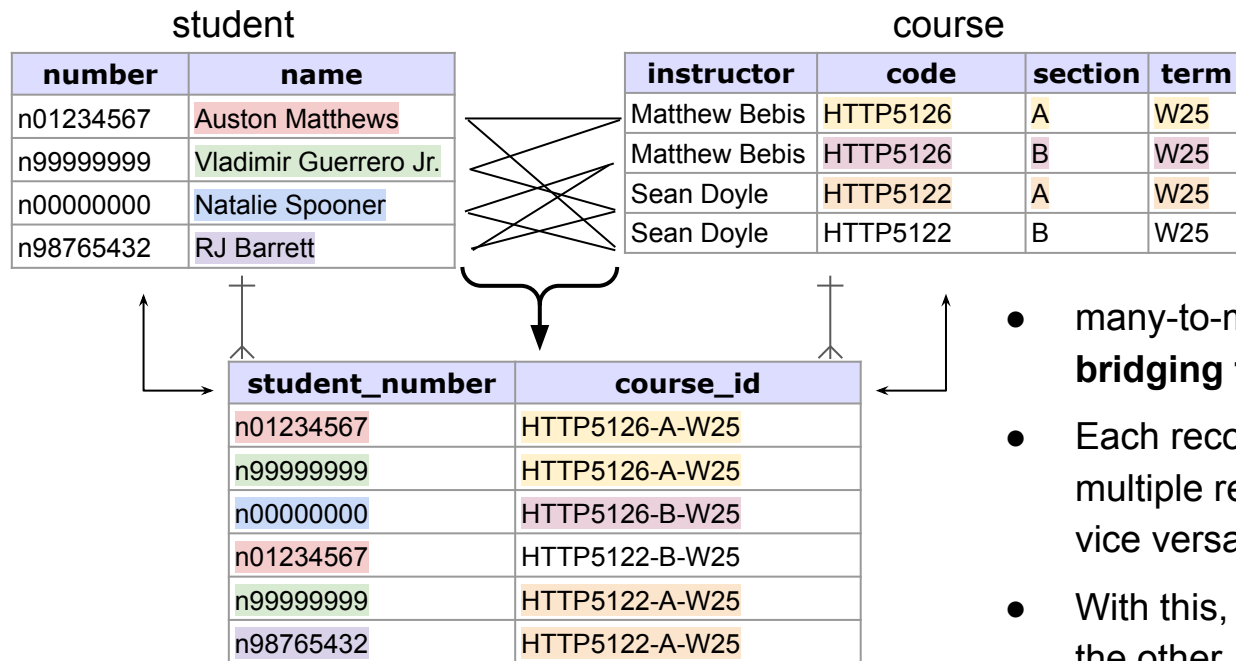


- many rows in a table connect to many rows in a different table
 - Example:
 - A student **HAS MANY** courses
 - A course **HAS MANY** students

Relationship Types / Bridge Tables¹

many-to-many relationship

*listing items in 1 cell indicates need for another table, connected with either a one-to-many or many-to-many relationship



- many-to-many relationships require a **bridging table** to connect them
- Each record in one table can relate to multiple records in another table, and vice versa.
- With this, neither table directly relates to the other, instead both relate through the bridge

Keys - Specify Foreign Keys¹

- **Foreign Key**: a column(s) in one table that **refers to the Primary Key** column in another table, to formalize the relationship between tables
- Foreign Keys can reference any type of primary key from other tables
 - i. Unique Data PK
 - ii. Composite PK
 - iii. id PK

i.

PK	
number	name
n01234567	Auston Matthews

Foreign Key	
number	address
n01234567	40 Bay St.

ii.

(Primary) Composite Key	
code	section
HTTP5126	A

(Foreign) Composite Key		
student_id	code	section
n01234567	HTTP5126	A

iii.

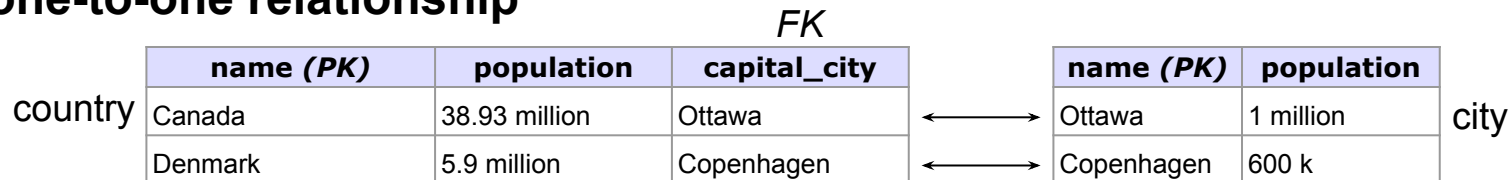
PK	
bus_id	route
1	508

FK	
bus_id	driver
1	Sean Doyle

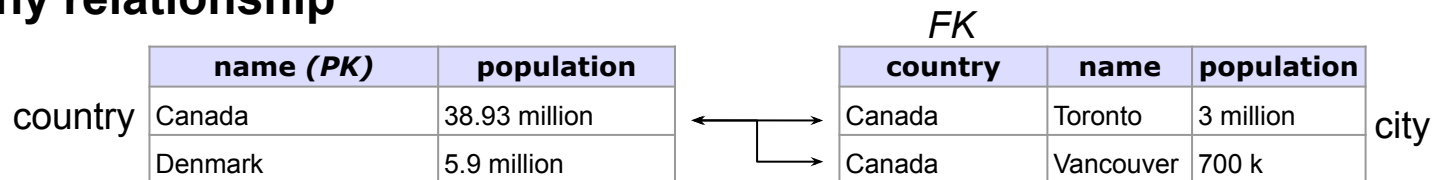
¹ [What are Foreign Keys?](#)

Keys - Specify Foreign Keys

one-to-one relationship



one-to-many relationship



many-to-many relationship *Composite PK* *That also acts as a FK

number (PK)	name
n01234567	Auston Matthews
n999999999	Vladimir Guerrero Jr.
n000000000	Natalie Spooner
n98765432	RJ Barrett

student_number	course_id
n01234567	1
n999999999	1
n000000000	2
n01234567	2

course_id (PK)	instructor	code	section	term
1	Matthew Bebis	HTTP5126	A	W25
2	Matthew Bebis	HTTP5126	B	W25
3	Sean Doyle	HTTP5122	A	W25
4	Sean Doyle	HTTP5122	B	W25

Keys - Naming Conventions

- **Primary Key**

- Should follow all column naming conventions (Week 2) *AND*
- Should be named: *id*

- **Foreign Key**

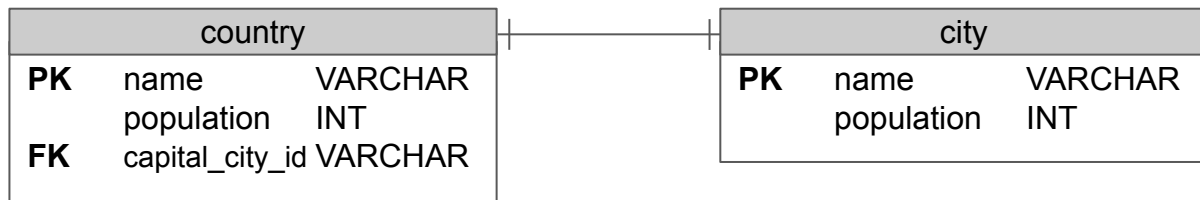
- Should follow all column naming conventions (Week 2) *AND*
- Should be named: table it is referring to, followed by *_id*

Primary Key		Foreign Key	
↓		↓	
<i>movie</i>		<i>director</i>	
id (PK)	title	director_id (FK)	
1	The Banshees of Inisherin	1	
2	The Truman Show	2	
3	Eternal Sunshine of the Spotless Mind	3	
4	The Dark Knight	4	

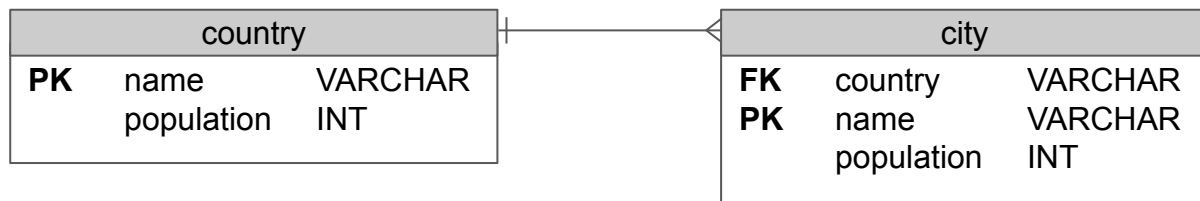
Primary Key	
↓	
id (PK)	name
1	Martin McDonagh
2	Peter Weir
3	Michel Gondry
4	Christopher Nolan

Entity Relationship Diagrams (ERD) *Evolution of Table Diagrams

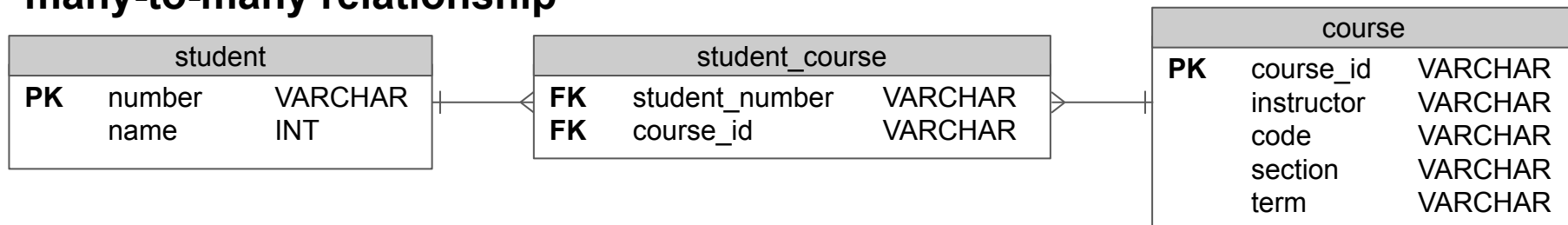
one-to-one relationship



one-to-many relationship

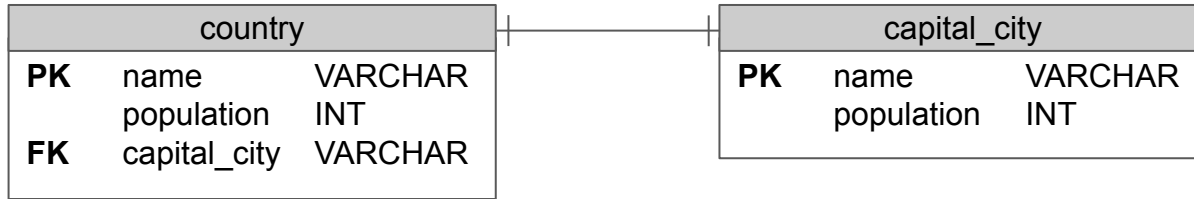


many-to-many relationship

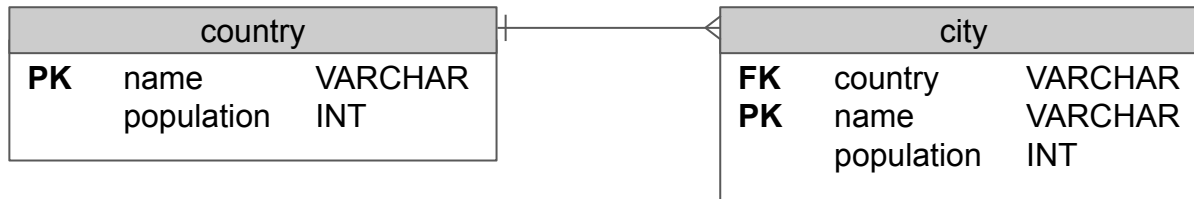


Multiple Relationship Types

one-to-one relationship

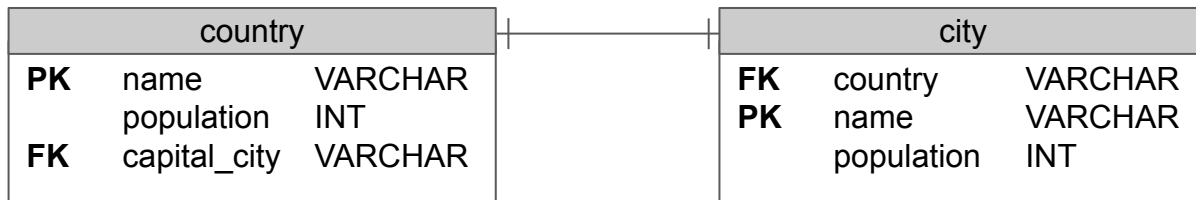


one-to-many relationship

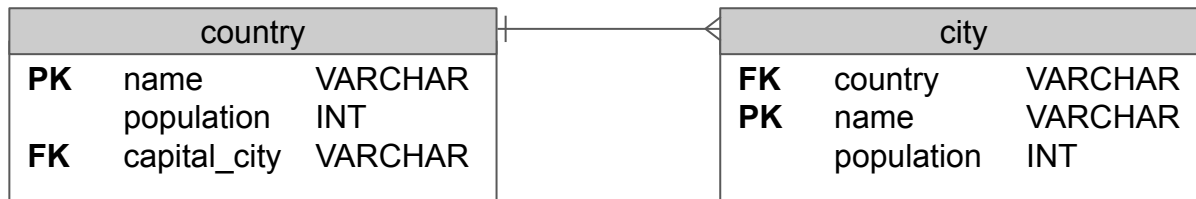


Multiple Relationship Types

one-to-one relationship

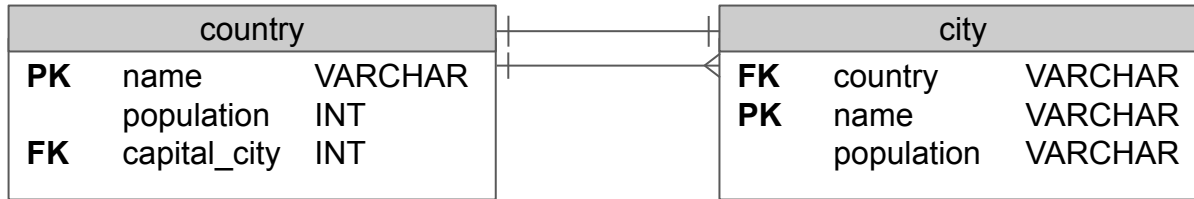


one-to-many relationship



Multiple Relationship Types

one-to-one relationship *AND* one-to-many relationship



- It is ok for 2 tables create multiple relationships with each other, as long as the tables pass the normalization rules

Table Constraints

- Constraints are used to specify rules and limit the data for columns in our tables
- This ensures the accuracy and reliability of the data in the table
- Violations of constraints abort queries
- Constraints can be column level or table level

Constraint Types¹

NOT NULL - Ensures that a column cannot have a NULL value

UNIQUE - Ensures that all values in a column are different

***PRIMARY** and **FOREIGN KEYS** are also constraints

PRIMARY KEY - Both **NOT NULL** & **UNIQUE**

NOT NULL¹

instructor		
PK	instructor_id	INT
NN	name	VARCHAR

UNIQUE²

instructor		
PK	instructor_id	INT
NN	name	VARCHAR
U	employee_id	VARCHAR

NOT NULL & UNIQUE

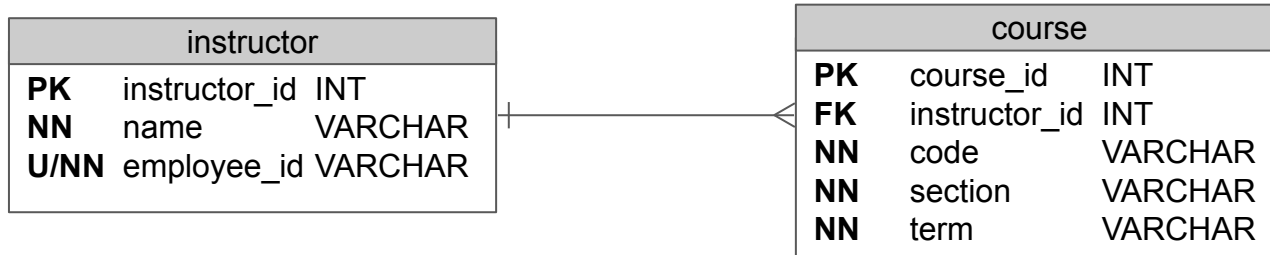
instructor		
PK	instructor_id	INT
NN	name	VARCHAR
U/NN	employee_id	VARCHAR

¹ [SQL Constraint NOT NULL](#)

² [SQL Constraint UNIQUE](#)

Entity Relationship Diagrams (ERD)

- Used to represent a single databases architecture
- ERDs have varying layouts but will generally show this information:
 - Entities (tables)
 - Fields (columns)
 - Data types
 - Relationships
 - Constraints



Refine your design with Guiding Principles¹

- **Reduce Redundancy**, duplicate information is bad
 - It wastes space and increases the likelihood of errors and inconsistencies
- **Correctness and Completeness**, no exceptions
 - Incorrect data leads to incorrect data driven decisions

A good database design is, therefore, one that:

- Divides your information into subject-based tables to reduce redundant data
- Creates relationships between tables to join information together as needed
- Helps support and ensure the accuracy and integrity of your information
- Accommodates data processing and reporting needs

¹ [Microsoft - DB Design Basics - What is good database design?](#)

Normalization Forms

Normalization is the process of organizing data in a database to eliminate redundancy and inconsistencies

Normalization can be achieved by applying each normal form step by step from the earliest normal forms to the latest

The normal forms are:

1NF - 1st Normal Form

2NF - 2nd Normal Form

3NF - 3rd Normal Form

BCNF - Boyce-Codd Normal Form

4NF - 4th Normal Form

ETNF - Essential Tuple Normal Form

5NF - 5th Normal Form

DKNF - Domain-Key Normal Form

6NF - 6th Normal Form

Normalized DBs

- There are differing opinions for when a database is considered normalized
- Some argue normalization occurs after 5NF
- It is best to assume a DB is normalized when it meets the standards of your use case
- 3NF is generally accepted as normalized, with BCNF used as more robust solution in rare cases where 2NF/3NF miss redundancies

Normal Forms

First Normal Form (1NF)

- Each attribute (column) has a unique name
- Domain of attributes must not change (same data type)
- Each row is uniquely identifiable (no duplicate rows)
- Each cell must have only a single value, non-divisible values (atomicity)

Second Normal Form (2NF) – Including 1NF

- Every column is dependent on the whole primary key

Third Normal Form (3NF) – Including 1NF, 2NF

- All non-key columns are functionally dependent on the primary key

Normal Forms

First Normal Form (1NF) / Example

- Each attribute has a unique name
- Domain of attributes must not change (same data type)
- Each row is uniquely identifiable (no duplicate rows)
- Each cell must have only a single value, non-divisible values (atomicity)

offering_id	instructor	code	section	term
1	Matthew Bebis	HTTP5126	A	F24
2	Matthew Bebis	HTTP5126	B	F24
3	Sean Doyle	HTTP5122	A	F24
4	Sean Doyle	HTTP5122	B	F24

offering_id	instructor_first_name	instructor_last_name	program_code	course_code	section	term	year
1	Matthew	Bebis	HTTP	5126	A	F	2024
2	Matthew	Bebis	HTTP	5126	B	F	2024
3	Sean	Doyle	HTTP	5122	A	F	2024
4	Sean	Doyle	HTTP	5122	B	F	2024

Normal Forms

Second Normal Form (2NF) – Including 1NF / Example

- Every column is dependent on the whole primary key

offering_id	instructor_first_name	instructor_last_name	program_code	course_code	section	term	year
1	Matthew	Bebis	HTTP	5126	A	W	2025
2	Matthew	Bebis	HTTP	5126	B	W	2025
3	Sean	Doyle	HTTP	5122	A	W	2025
4	Sean	Doyle	HTTP	5122	B	W	2025



offering_id	instructor_first_name	instructor_last_name	program_code	course_code	section
1	Matthew	Bebis	HTTP	5126	A
2	Matthew	Bebis	HTTP	5126	B
3	Sean	Doyle	HTTP	5122	A
4	Sean	Doyle	HTTP	5122	B



term	year	start_date	end_date
F	2024	2024-09-01	2024-12-17
W	2025	2025-01-03	2025-04-21
S	2025	2025-05-06	2025-08-18

Normal Forms

Third Normal Form (3NF) – Including 1NF, 2NF / Example

- All non-key columns are functionally dependent on the primary key

offering_id	instructor_first_name	instructor_last_name	program_code	course_code	section
1	Matthew	Bebis	HTTP	5126	A
2	Matthew	Bebis	HTTP	5126	B
3	Sean	Doyle	HTTP	5122	A
4	Sean	Doyle	HTTP	5122	B

term	year	start_date	end_date
F	2024	2024-09-01	2024-12-17
W	2025	2025-01-03	2025-04-21
S	2025	2025-05-06	2025-08-18

instructor_first_name	instructor_last_name
Matthew	Bebis
Matthew	Bebis
Sean	Doyle
Sean	Doyle

program_code
HTTP
HTTP
HTTP
HTTP

Normal Forms

Third Normal Form (3NF) – Including 1NF, 2NF / Example

- All non-key columns are functionally dependent on the primary key

offering_id	code	section	name
1	5126	A	Database Design & Development
2	5126	B	Database Design & Development
3	5122	A	Front-End Web Development
4	5122	B	Front-End Web Development

instructor_id	first_name	last_name
n024681357	Matthew	Bebis
n753186420	Sean	Doyle

code	name
HTTP	Web Development
HUMA	Humanities
IDSN	Industrial Design

term	year	start_date	end_date
F	2024	2024-09-01	2024-12-17
W	2025	2025-01-03	2025-04-21
S	2025	2025-05-06	2025-08-18

Normal Forms

Third Normal Form (3NF) – Including 1NF, 2NF / Example

- All non-key columns are functionally dependent on the primary key

offering_id	section	course_code	program_code	instructor_id	term
1	A	5126	HTTP	n024681357	W2025
2	B	5126	HTTP	n024681357	W2025
3	A	5122	HTTP	n753186420	W2025
4	B	5122	HTTP	n753186420	W2025

code	name
5126	Database Design & Development
5122	Front-End Web Development

instructor_id	first_name	last_name
n024681357	Matthew	Bebis
n753186420	Sean	Doyle

code	name
HTTP	Web Development
HUMA	Humanities
IDSN	Industrial Design

term	year	start_date	end_date
F	2024	2024-09-01	2024-12-17
W	2025	2025-01-03	2025-04-21
S	2025	2025-05-06	2025-08-18

Unnormalized Data

offering_id	instructor	code	section	term
1	Matthew Bebis	HTTP5126	A	F24
2	Matthew Bebis	HTTP5126	B	F24
3	Sean Doyle	HTTP5122	A	F24
4	Sean Doyle	HTTP5122	B	F24

Normalized Data

offering_id	section	course_code	program_code	instructor_id	term
1	A	5126	HTTP	n024681357	W2025
2	B	5126	HTTP	n024681357	W2025
3	A	5122	HTTP	n753186420	W2025
4	B	5122	HTTP	n753186420	W2025

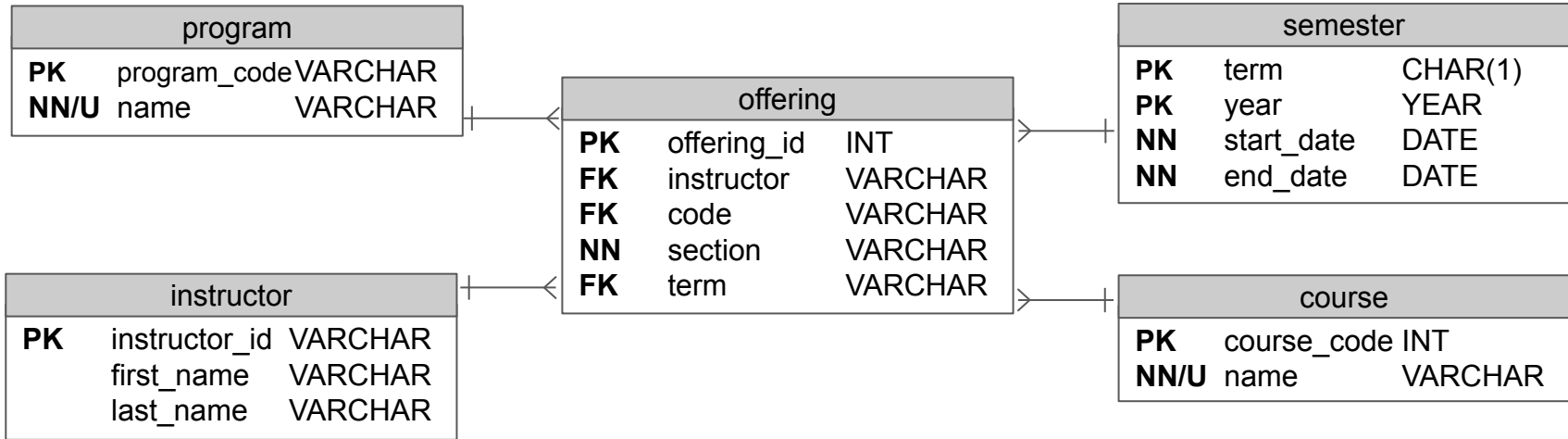
course_code	name
5126	Database Design & Development
5122	Front-End Web Development

instructor_id	first_name	last_name
n024681357	Matthew	Bebis
n753186420	Sean	Doyle

program_code	name
HTTP	Web Development
HUMA	Humanities
IDSN	Industrial Design

term	year	start_date	end_date
F	2024	2024-09-01	2024-12-17
W	2025	2025-01-03	2025-04-21
S	2025	2025-05-06	2025-08-18

Normalized Data ERD



Unnormalized Data

customer_name	product_name	supplier	supplier_location	price	brand	order_id	email
John	Laptop	XYZ Electronics	Toronto	800	Dell	1	john@gmail.com
Jane	Smartphone	ABC Gadgets	Montreal	600	Samsung	2	jane@gmail.com
John	Camera	XYZ Electronics	Toronto	300	Canon	3	john@gmail.com
Alice	Chair	XYZ Furniture	Vancouver	100	Herman Miller	4	alice@gmail.com

Normalized Data

order_id (PK)	email (FK)
1	john@gmail.com
2	jane@gmail.com
3	john@gmail.com
4	alice@gmail.com

order_id (PK)	product_id (PK)
1	1
2	2
2	3
3	4
4	5

product_id(PK)	name	price	brand	supplier_id (FK)
1	Laptop	800	Dell	1
2	Smartphone	600	Apple	2
3	Smartphone	600	Samsung	2
4	Camera	300	Canon	1
5	Chair	100	Herman Miller	3

email (PK)	name
john@gmail.com	John
jane@gmail.com	Jane
alice@gmail.com	Alice

supplier_id (PK)	name	location
1	XYZ Electronics	Toronto
2	ABC Gadgets	Montreal
3	XYZ Furniture	Vancouver

Normalized Data ERD

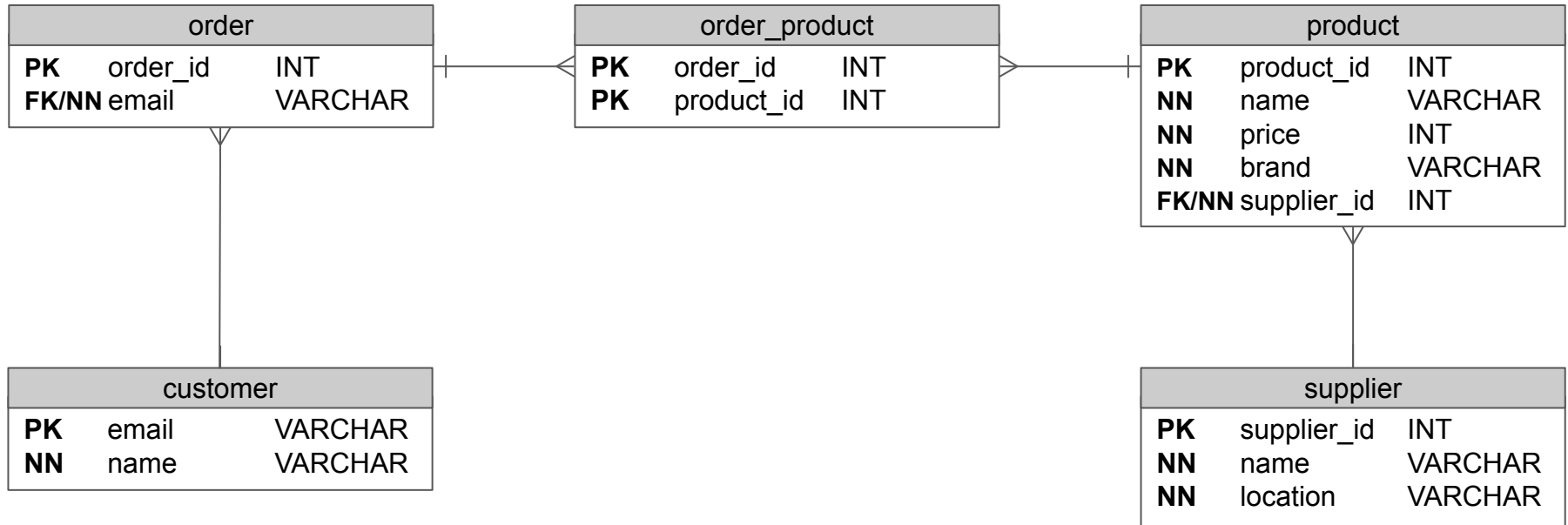


Table Relationships

- Determining the relationships between tables helps you ensure that you have the right tables and columns
- When a one-to-one or one-to-many relationship exists, the tables involved need to share a common column or columns
- When a many-to-many relationship exists, a third table is needed to represent the relationship.

Relationships will fall into 1 of 3 categories

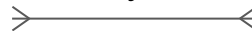
one-to-one relationship



one-to-many relationship



many-to-many relationship



Bridging Tables

AKA Junction or Intersection Tables

- Depending on your databases need, bridging tables can store:
 - just the relationship information
- OR**
- other related information for each unique many-to-many combination
- Depending on the situation we can use different naming styles for the tables

student_number	course_id
n01234567	HTTP5126-A-W25
n999999999	HTTP5126-A-W25
n000000000	HTTP5126-B-W25
n01234567	HTTP5126-B-W25
n999999999	HTTP5122-A-W25
n98765432	HTTP5122-A-W25

student_course
Bridge table

student_number	course_id	date_enrolled
n01234567	HTTP5126-A-W25	2024-09-11
n999999999	HTTP5126-A-W25	2024-10-02
n000000000	HTTP5126-B-W25	2024-11-01
n01234567	HTTP5126-B-W25	2024-11-12
n999999999	HTTP5122-A-W25	2024-11-29
n98765432	HTTP5122-A-W25	2024-12-02

enrollment
Bridge table

Why use Bridging Tables

- many-to-many relationships require a **bridging table** to relate them, if not...

- ☐ Repeated data is a data redundancy
- ☐ Compromises data integrity
- ☐ Inefficient querying

number	name	course_id
n01234567	Auston Matthews	HTTP5126-A-W25
n99999999	Vladimir Guerrero Jr.	HTTP5126-A-W25
n00000000	Natalie Spooner	HTTP5126-B-W25
n01234567	Auston Matthews	HTTP5126-B-W25
n99999999	Vladimir Guerrero Jr.	HTTP5122-A-W25
n98765432	RJ Barrett	HTTP5122-A-W25

instructor	code	section	term	student
Matthew Bebis	HTTP5126	A	W25	Auston Matthews
Matthew Bebis	HTTP5126	A	W25	Vladimir Guerrero Jr.
Matthew Bebis	HTTP5126	B	W25	Natalie Spooner
Matthew Bebis	HTTP5126	B	W25	RJ Barrett

Keys - Specify Foreign Keys

- Purpose of foreign keys is to create a relationship between two tables that:
 - Maintain Consistency
 - Ensure values in one table correspond to valid rows in another table, preventing data inconsistencies
 - Can Prevent Invalid Data
 - Blocks attempts at adding data with foreign key that does not have matching primary key
 - Can Enable Cascading Actions
 - Changes in the parent table automatically apply to the related rows in the child table

Table Constraints

- Rules and conditions applied to a columns to maintain data integrity, accuracy, and consistency
- Data must meet specific criteria
- Safeguards against data anomalies, errors, and inconsistencies
- Constraints can be column level or table level

NOT NULL (NN) - Ensures that a column cannot have a NULL value

UNIQUE (U) - Ensures that all values in a column are different

***PRIMARY (PK)** and **FOREIGN KEYS (FK)** are also constraints

PRIMARY KEY - Both **NOT NULL** & **UNIQUE**

Entity Relationship Diagrams (ERD)

- Used to represent a single databases architecture
- ERDs have varying layouts but will generally show this information:
 - Entities (tables)
 - Fields (columns)
 - Data types
 - Relationships
 - Constraints



Normal Forms

First Normal Form (1NF)

- Each attribute (column) has a unique name
- Domain of attributes must not change (same data type)
- Each row is uniquely identifiable (no duplicate rows)
- Each cell must have only a single value (atomicity)

Second Normal Form (2NF) – Including 1NF

- Every column is dependent on the whole primary key

Third Normal Form (3NF) – Including 1NF, 2NF

- All non-key columns are functionally dependent on the primary key